Anatomy of a Deephaven Application

Oct 20, 2026 · @Taylor

A Deephaven application is one worker's scripts, the update graph they build, the threads and clients that feed and read it, and whatever it writes out; this doc maps all of it in space and in time.

What we were missing

The biggest gap was control over time: a worker can run additional update graphs, including an event-driven one that advances only when told, which is exactly a framework-owned checkpoint. Everything else below fills in hooks, inputs, outputs and operations we had not used.

Feature	What it does	Framework meaning	Status
Event-driven update graph	A second named update graph that advances one step per `requestRefresh()`	A synchronizer can own its own clock instead of the shared ~1 s cycle	**Verified** (built from Python and driven: three pushes waited until one refresh delivered them as a single step)
Multiple update graphs per worker	Each execution context can carry its own graph (periodic or event-driven)	Resource isolation between parts of one app	**Docs**

Feature	What it does	Framework meaning	Status
Server executors	`deephaven.server.executors.submit_task("serial" or "concurrent", fn)` runs Python work on JVM server threads	Deferred work from listeners without your own thread; "serial" for anything taking the exclusive lock	**Verified** (both executors exist; a task ran)
Update-graph locks	`deephaven.update_graph` shared and exclusive locks, `auto_locking_ctx`	The rule for changing the graph from your own threads	**Docs**
Execution contexts	`get_exec_ctx()` (the script session's), `make_user_exec_ctx(freeze_vars=...)` for deferred work	Per thread: only the script session and the thread that started the server hold the real one, so capture it there and pass it to threads you start (Verified); user contexts can pin variable values	**Docs** (plus our rung 4)
Liveness scopes	`liveness_scope()` releases every node created inside it when it closes	Explicit cleanup of subscriptions and derived tables, instead of relying on garbage collection	**Docs**

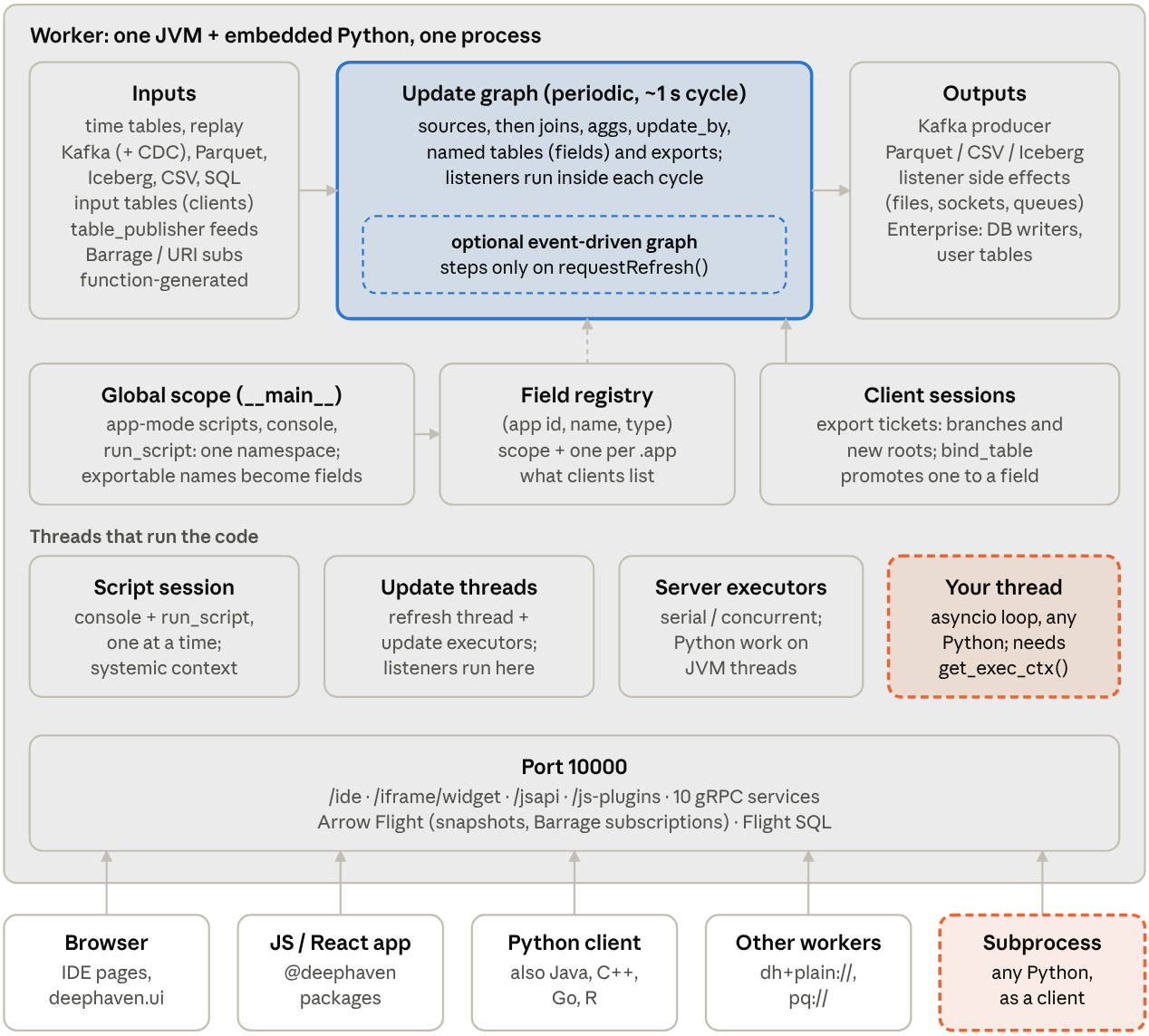
Feature	What it does	Framework meaning	Status
Systemic object marking	Marks objects so errors in them count as system errors	Error attribution in shared workers	**Docs**
Function-generated tables	`function_generated_table(fn, refresh_interval_ms or source_tables)` recomputes a table from Python on a timer or trigger	A polling input without your own thread	**Docs**
Ring tables, `DynamicTableWriter`, `await_update`, `snapshot_when`, synchronize tables, partitioned `transform`	Bounded history, an older writer API, waiting on a refresh, sampled snapshots, aligning sources, per-key subgraphs	Inputs, windows and per-key fan-out	**Docs**
Replay	`TableReplayer` replays historical data as if live	Back-testing and catch-up from history	**Docs**
Data I/O	Parquet, CSV, Iceberg, Kafka with CDC, SQL (`experimental.sql`), ADBC/ODBC, S3, a pluggable table data service	Sources and sinks beyond Kafka and Python feeds	**Docs**
In-worker Barrage sessions	`deephaven.barrage.barrage_session(...)` subscribes to another server's tables from inside a worker	Community's equivalent of `pq://`	**Docs**; URI form **Verified**
Custom object types	`ObjectType` / `BidirectionalObjectType` with a `MessageStream`, plus a `JsPlugin`; plugin authorization	Your own typed RPC to browsers and clients: how deephaven.ui itself works	**Docs**
Query library	`import_class`, `import_static`, `import_package`	Java classes callable from formulas	**Docs**
Configuration	`get_configuration()`, server properties, update-graph cycle settings	Reading and tuning server dials from the app	**Docs**

Feature	What it does	Framework meaning	Status
deephaven.ui routing	`router`, routes, `use_navigate`, `use_params`, `use_query_param`	Multi-page apps with URLs	**Docs**
deephaven.ui shared state	`create_global_state`, `create_user_state`	State shared across sessions or per user, not only per render	**Docs**
deephaven.ui thread hooks	`use_render_queue`, `use_execution_context`, `use_table_listener`, `use_liveness_scope`, `use_send_event`	Updating a UI from other threads; listener and cleanup hooks inside components	**Docs**
Clients	Python, Java, JS, C++, Go, R, Flight SQL (and JDBC/ODBC on Enterprise), MCP	More ways in than we used	**Docs**
Community auth	Anonymous, pre-shared key, username/password, mTLS, Keycloak	Production login options	**Docs**
Enterprise PQ management	Git-sourced scripts, worker configuration, PQ XML schema, modularized queries, replay queries	Answers our open question: PQ scripts can come from Git	**Docs**
Enterprise data	Data Import Server, binary-log loggers and tailers, Kafka and **Solace** streaming import, SQL Server CDC, lastBy ingester, Derived Table Writer, user tables, merge and validation	Durable state and ingestion; corrects our earlier "no built-in Solace"	**Docs**
Enterprise operations	etcd, controller, configuration server, authentication service, table data cache proxy, log aggregator, Web API service, Envoy; PQ redundancy and 24x6 uptime; PQ, table and plugin ACLs; LDAP, SAML, IAP	The platform around each worker	**Docs**

Space: the application inside one worker

Everything the app does happens inside one worker process, around its update graphs (one periodic graph by default, plus any event-driven graphs you add); clients, other workers and your own subprocesses all reach it through one port.

One worker: inputs feed one update graph; clients reach it through port 10000



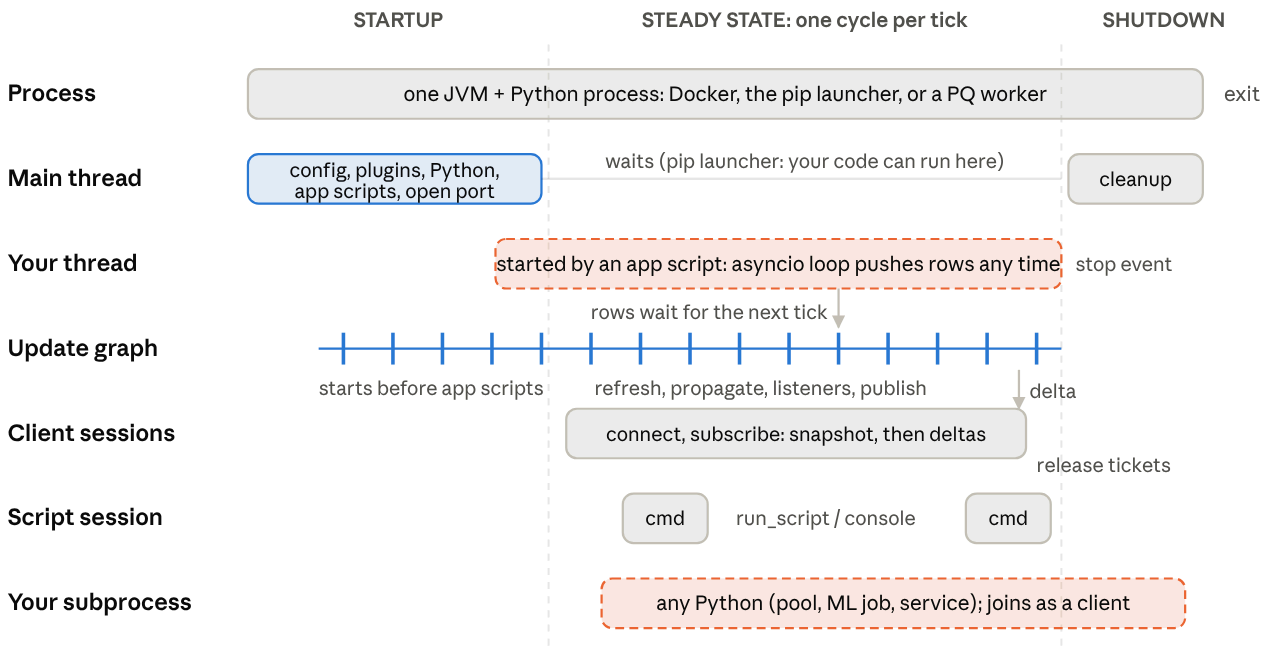
space · a full application inside one worker, its threads and its clients

Blue is the engine's graph; dashed orange is code you run outside the engine's control: a thread inside the worker needs an execution context, while a subprocess can only be a client.

Time: one application's lifecycle

Startup runs once, on whichever thread starts the server (the main thread in a stock worker), and builds the graph; from then on the update graph's ticks drive everything, and shutdown is only as orderly as your own cleanup.

Startup builds the graph once; every cycle then feeds listeners and clients



Startup order from our server logs; tick spacing illustrative. Clients can connect only after the port opens.

time · startup, steady-state cycles and shutdown across threads and processes

Read a column top to bottom for one moment in time. Work from your thread waits for the next tick, flows through the graph, and reaches clients as one consistent delta.

Who runs what: threads, processes and contexts

The engine owns most threads; your code runs on the thread that starts the server (application-mode scripts, or an embedding host's server thread), on the script session, inside listeners on update threads, or on threads and processes you start, and only those last ones do not get an execution context for free. Thread names below are from our introspection report on Community 42.6. **Verified.**

Thread or process	Controlled by	Runs	Execution context	Lock
The thread that starts the server	The launcher: Docker's server main, or whichever thread calls `Server(...).start()` with the pip server (it need not be the main thread)	Startup, including application-mode scripts	Systemic	As each operation needs
`DeephavenApiServer-Scheduler-Serial-1`	Script session	Console and `run_script` commands, one at a time	Systemic	As each operation needs
`PeriodicUpdateGraph.DEFAULT.refreshThread`	Update graph	The cycle: refresh sources, start propagation	Engine	Exclusive, for the cycle
`PeriodicUpdateGraph-updateExecutor-N` (12 here)	Update graph	Propagation in dependency order, including **your listeners**	Engine	Inside the cycle
`OperationInitializationThreadPool-initializationExecutor-N`	Engine	Parallel initial computation of new table operations	Engine	Shared, as needed
`DeephavenApiServer-Scheduler-Concurrent-N`	Server	Client requests; `submit_task("concurrent", fn)` work	Server	Shared, as needed
Jetty `qtp...` threads	Web server	HTTP, gRPC and websocket transport	None	None
An event-driven update graph	Whoever calls `requestRefresh()`	Its own cycles, for tables built under a context bound to it	Its own	Its own graph's
Your Python threads (our `ex4-asyncio`, or an embedding host's main thread)	You	Anything: feeds, timers, sockets, an asyncio loop	**Static-only by default.** Enter a context captured on a thread that holds the real one	Take one explicitly before touching the graph
Your subprocesses	You	Anything, in a separate interpreter	No access to the worker except as a client (`pydeephaven`) or through your own IPC	Not applicable
Enterprise controller and dispatcher	The platform	Start, stop and place PQ workers	Not applicable	Not applicable

All Python on every thread shares one GIL. Java threads that call into Python appear in `threading.enumerate()` as `java-...` entries.

Endpoints and client services

A Community worker exposes everything on one port, 10000: web pages, the JS API, ten gRPC services and Arrow Flight, which carries Barrage.

Endpoint	What it serves	Status
`/ide/`	The web IDE: console, panels, notebooks	**Verified**
`/iframe/widget/?name=<field>`	One field as a standalone page, such as a deephaven.ui dashboard	**Verified**
`/jsapi/dh-core.js`	The JS API, loaded by browser clients	**Verified**
`/js-plugins/`	JS plugin manifest and bundles (deephaven.ui, plotly-express)	**Verified**
`SessionService`	Login, session keep-alive, export and release of tickets	**Verified** (stub present)
`ConsoleService`	Script sessions and `run_script` / `runCode`	**Verified**
`TableService`	Table operations on tickets (sort, filter, join, …)	**Verified**
`ApplicationService`	`ListFields`: the field registry, with live updates	**Verified**
`ObjectService`	Message streams for plugin objects such as deephaven.ui elements	**Verified** (stub present)
`InputTableService`, `PartitionedTableService`, `HierarchicalTableService`	Writes to input tables; partitioned and tree/rollup operations	**Verified** (stubs present)

Endpoint	What it serves	Status
`StorageService`	Files the web UI saves: notebooks and layouts	**Verified** (stub; the storage directories it needs)
`ConfigService`	Server and auth configuration values for clients	**Verified** (stub present)
Arrow Flight `DoGet` / `DoPut` / `DoExchange`	Snapshots, uploads and Barrage subscriptions	**Verified** (snapshots and subscriptions), **Docs** (uploads)
Flight SQL	SQL clients and JDBC/ODBC drivers	**Docs**
`dh+plain://host:port/scope/<name>`, `.../app/<id>/field/<name>`	URIs resolved by `deephaven.uri.resolve`, as Barrage subscriptions	**Verified**
`pq://<PQ>/scope/<name>`	Enterprise URI to another PQ's field	**Docs**
Your own ports	Anything a Python thread in the worker listens on	Reported by our introspector on Linux

On Enterprise, clients come in through the platform's front door (Envoy and the Web API service) and the controller locates each PQ's worker. **Docs.**

Persistence, serialization and shutdown

Nothing in a worker's memory survives it; an application persists only what it writes out, and rebuilds from that in its next startup script.

What persists

- **Your outputs (Community):** Parquet, CSV and Iceberg writes, a Kafka producer, and anything a listener writes to files, sockets or databases.

- **The server's own files:** notebooks and layouts saved by the web UI (the storage service's directories), plus its config, cache and data directories. **Verified** (directories logged at startup; the pip server failed until they were valid on Windows).
- **Enterprise:** database tables written through loggers and the Data Import Server or the Derived Table Writer, user tables, input tables listed under the database, and merges from intraday to historical. **Docs.**
- **The definition, not the instance:** a PQ's script, schedule and settings live in the controller's store, not in the worker. **Docs.**

What crosses the wire

- **Tables** travel as Arrow record batches: Flight for snapshots, Barrage for live change sets.
- **deephaven.ui** documents and callbacks travel as JSON-RPC over an object message stream; tables inside them travel as tickets.
- **Custom plugins** send whatever bytes they define, plus references to server objects.
- **Python state is yours.** There is no deserialize hook; pickle or write it somewhere durable and reload it at startup.

Shutdown

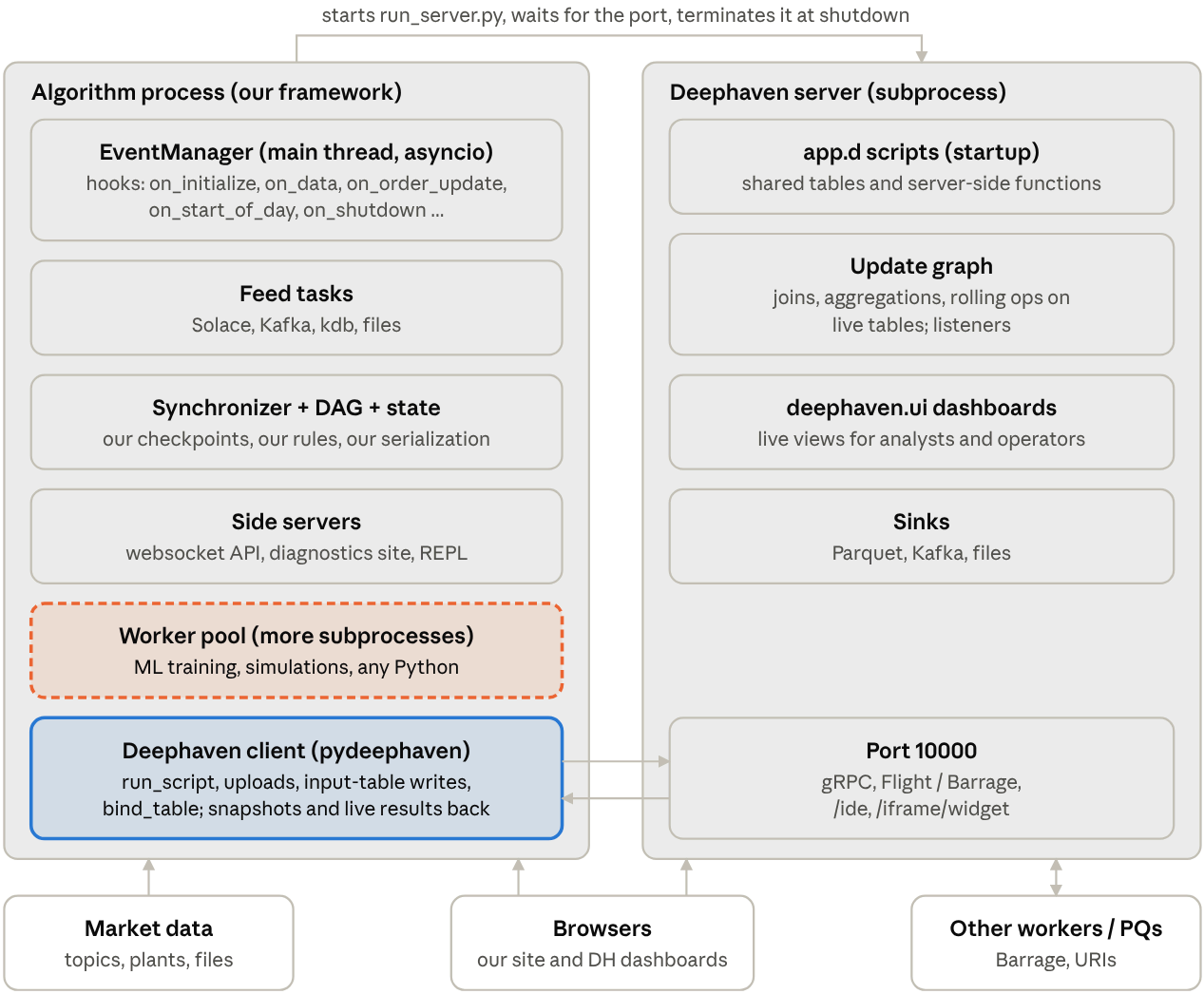
- **A worker stops when its process stops**: Docker stop, the pip server's process exiting, or an Enterprise scheduled stop.
- **Java-side cleanup is ordered** by the JVM's shutdown manager; we found no documented Python shutdown hook. With the pip server your launcher is `__main__`, so its own `finally` blocks and `atexit` handlers are the natural place for cleanup on a normal exit. **Unverified**; a hard kill skips all of it.
- **Clean up deliberately**, in this order: set stop events for your threads and tasks, stop listener handles (or close a liveness scope), flush sinks, release client tickets and close sessions. Our tests showed that a leaked listener or task survives until the worker restarts.

Deephaven as a service in our framework

Inverting the relationship avoids the lifecycle and shared-interpreter problems of running a framework inside a worker: our process owns the event loop, its own checkpoint schedule and the lifecycle, and starts Deephaven beside it as a service it reaches only as a

client. The data it reads still advances on Deephaven's ~1 s cycle, so this pattern does not give us the engine's clock. The next section shows the same service hosted on a thread instead.

Our framework hosts Deephaven as a service it starts, talks to, and stops



Upper arrow: commands, uploads and input-table writes. Lower arrow: snapshots and live results. Each side keeps its own clock, lifecycle and failure domain; they meet only at port 10000.

hosting · the algorithm framework runs the Deephaven server as a subprocess

What each side does best:

- **Our framework** keeps its asyncio loop, its own checkpoint rules, side servers, subprocess pools and serialization.
- **Deephaven** runs shared table logic on live data, serves dashboards, sinks to Parquet or Kafka, and is reachable by other workers and browsers.

- **The boundary is one port.** A Deephaven failure cannot take down the framework, and the framework's memory and threads stay out of the worker.

We ran this pattern end to end (`tests/host_service_demo.py`, native Windows). The host started a second server as a subprocess, which was ready in 10.9 s. It deployed a live aggregate with `run_script` and read it at three checkpoints on its own 1.5 s clock. Per-symbol counts went 6, 21, 26: they arrive in whole engine cycles, so they step unevenly against our clock. A process pool ran unrelated computation alongside, and shutdown closed the session and stopped the server. **Verified.**

One caveat surfaced: on Windows `terminate()` is a hard kill (exit code 1), so the server gets no chance to clean up. For a graceful stop, have the server shut itself down on request, for example a `run_script` that tells the launcher to exit, and flush sinks first.

The host side, condensed from the test:

```
class DeephavenService:
    """Start the server as a subprocess, connect as a client, stop it at
shutdown."""
    def start(self, timeout=180):
        cmd = [self.server_python, "server/run_server.py", "--port",
str(self.port), "--no-app"]
        self.proc = subprocess.Popen(cmd)          # JAVA_HOME must point at
Java 17+
        deadline = time.time() + timeout
        while True:                                # port opens after startup
finishes
            try:
                self.session = Session(host="localhost", port=self.port,
auth_type="Anonymous")
                return self
            except Exception:
                if self.proc.poll() is not None or time.time() > deadline:
                    raise RuntimeError("Deephaven did not come up")
                time.sleep(2)

    def stop(self):
        self.session.close()                       # releases this client's
tickets
        self.proc.terminate()                      # hard kill on Windows
        self.proc.wait(timeout=30)

class Algorithm:
```

```
    async def on_initialize(self):
        self.dh.session.run_script(
            "ticks = time_table('PT0.1s').update([...])\n"
            "summary = ticks.agg_by([agg.count_('N'), agg.avg('AvgPx = Px')],
by='Sym')")
        self.summary = self.dh.session.open_table("summary")   # live on the
server

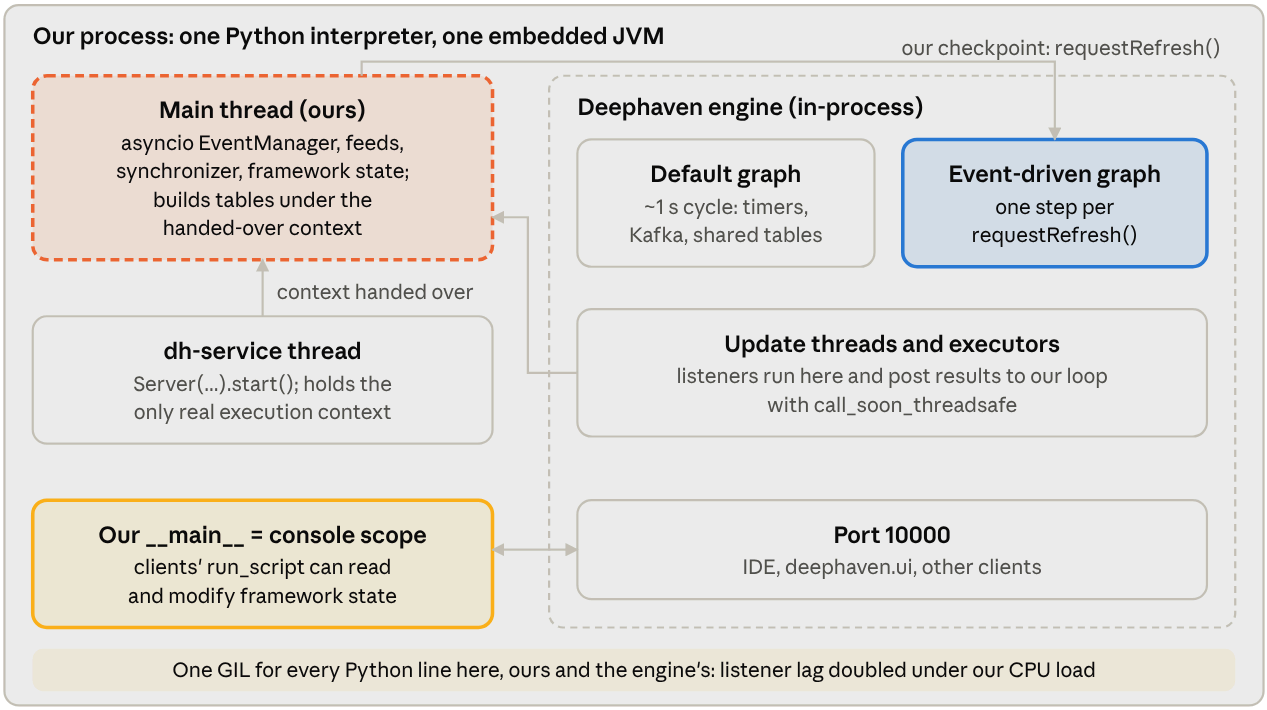
    async def on_checkpoint(self, k):                            # OUR clock
        df = self.summary.to_arrow().to_pandas()                 # fresh
snapshot
        side = await loop.run_in_executor(self.pool, arbitrary_work, 200_000)

async def main():
    dh = DeephavenService(server_python, 10001).start()
    with ProcessPoolExecutor(2) as pool:
        try:
            algo = Algorithm(dh, pool)
            await algo.on_initialize()
            for k in range(3):
                await asyncio.sleep(1.5)
                await algo.on_checkpoint(k)
        finally:
            dh.stop()
```

Thread or subprocess: hosting Deephaven inside our process

Deephaven does not insist on the main thread: the pip server started and served clients from a background thread of our own process, so the subprocess can become a thread, at the cost of sharing one interpreter, one failure domain and our own globals.

Embedded: Deephaven on a thread of our process, our synchronizer on its clock



embedded hosting · one process, our main loop, Deephaven on a thread

The win is the blue box: our synchronizer decides when the event-driven graph advances. The costs are in amber: one shared interpreter, and our module globals open to any client with console access.

Results from `tests/embedded_thread_demo.py` and `tests/probe_event_driven_clock.py` (Community 42.6, native Windows). **Verified.**

Question	Result
Can the server start off the main thread?	Yes: on a thread named `dh-service`, ready in 9.4 s; clients connected normally
Does our event loop keep running during startup?	Yes, at about half speed: 89 ticks of a 50 ms sleep in 9.4 s
Can the host use tables in-process, with no client?	Yes, under the context captured on the server's thread. The main thread's own context is static-only, and a ticking table failed there

Question	Result
Can in-process listeners feed our event loop?	Yes: a listener posting to asyncio with `call_soon_threadsafe` delivered 40 rows in 4 s, one batch per engine cycle
Does CPU-heavy host Python affect the engine's Python?	Yes: median listener lag rose from 36.6 to 74.8 ms while the main thread burned CPU
What is the console scope?	Our own `__main__`: a client's `run_script` read and modified the host's framework state
Can we restart the server in the same process?	No: a second `Server(...).start()` failed with "Cannot create more than one instance of the server"
Does the process exit when our code finishes?	Yes, promptly, once our main coroutine returned (server on a daemon thread; an orderly server shutdown was not tested)
Can our synchronizer own the clock?	Yes: on an event-driven graph nothing moved until `requestRefresh()`, and one refresh delivered three pushes as one step

The three ways to combine our framework with Deephaven:

	Framework inside a worker	Deephaven on a thread of our process	Deephaven as a subprocess or remote service
Owns the main loop	Deephaven	Us	Us
Checkpoints	The default ~1 s cycle, or an event-driven graph	An event-driven graph our synchronizer steps directly	Snapshots and subscriptions on Deephaven's cycle
Data path	In-process	In-process, no network hop	Network: Barrage subscriptions and snapshots
Python interpreter	Shared with the engine	Shared: listener lag doubled under our load	Separate

	Framework inside a worker	Deephaven on a thread of our process	Deephaven as a subprocess or remote service
Failure domain	Shared	Shared: a crash takes both down	Separate
Console exposure	Our code lives in the worker's scope	Our `__main__` is the console scope	Only what we publish
Restart	A new worker	Only by restarting our process	Restart the subprocess
On the Enterprise plant	A PQ	Not applicable: PQ workers are processes the platform starts	A client of PQs

Use the thread when in-process latency and owning the clock matter and we control who can connect: PSK authentication, the console disabled, framework state kept out of `__main__`. Use the subprocess when isolation matters. On the Enterprise plant, PQs stay separate workers either way.

Confidence and sources

Verified means observed on Deephaven Community 42.6 (pip server, native Windows, Java 17) or read in its installed source; **Docs** means stated in Deephaven documentation; **Unverified** means our understanding.

- Tests: `tests/probe_concepts.py`, `tests/probe_introspection.py`, `tests/probe_runtime_features.py`, tests/host_service_demo.py, tests/embedded_thread_demo.py, tests/probe_event_driven_clock.py, the ledger VERIFIED_FACTS.md, and the experiment ladder in `examples/`.
- Core documentation, including Execution Context
- Enterprise documentation
- deephaven.ui documentation
- Companion docs: Deephaven Concepts: Scopes, Fields, Tickets and Barrage and Deephaven as an Application Framework: Findings and Guidance